

Organización del Computador



Entrada/Salida: Interrupciones

2024

¿Qué es una interrupción?

Definición

Es un evento que interrumpe la ejecución de un programa en ejecución (proceso) para pasar a ejecutar una rutina denominada servicio de interrupción

Una interrupción da la apariencia que un sistema realiza varias tareas simultáneamente, pero la CPU sólo puede ejecutar una instrucción cada ciclo de máquina (¡por núcleo!)

El proceso anterior se conoce como **subrutina**

¿Qué es una interrupción?

La diferencia de un sistema con interrupciones es que la interrupción no se genera con una instrucción `bl` o `bx`, sino en respuesta a una llamada por **HARDWARE** o por **SOFTWARE**.

El programa que se ejecuta cuando se genera una interrupción se denomina **RUTINA DE ATENCIÓN A INTERRUPCIÓN (ISR)**.

Fuentes de interrupción

Hardware

- Señales eléctricas enviadas desde los dispositivos de hardware,
- Generalmente concentradas en un controlador externo que las prioriza y envía a la CPU secuencialmente de acuerdo a este criterio.
- Características: Asincrónicas, no determinísticas

Software

- Si la CPU tiene un su set de instrucciones una o más instrucciones definidas para tal fin, ejecutando esa instrucción se genera una interrupción.
- Característica: Son determinísticas, ya que se producen en un determinado y preciso momento dentro del programa.
- La instrucción apropiada se intercala donde se desea genera la interrupción.

Fuentes de interrupción

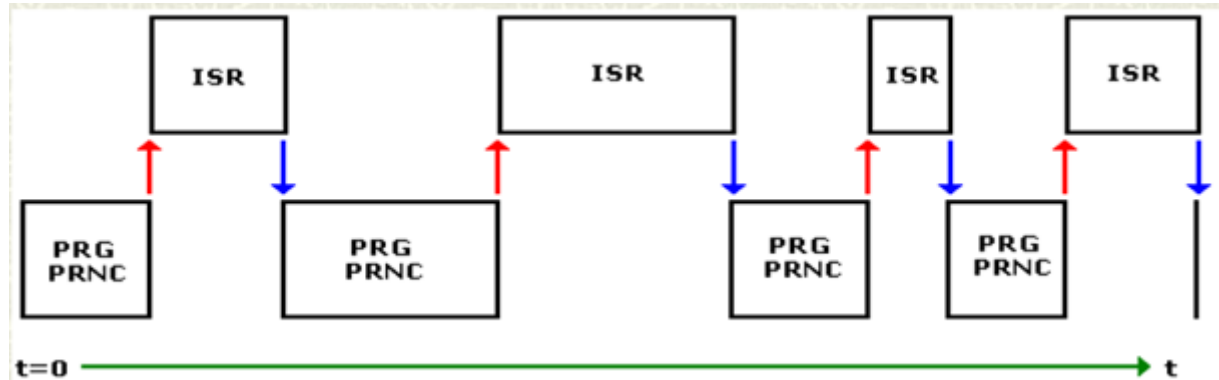
Internas

- Se conocen por lo general como excepciones.
- Son generadas por la propia CPU como consecuencia de una situación que le impide completar la ejecución de la instrucción en curso.
- Los ejemplos dependen de la CPU que se analice, pero por lo general encontrar un OPCODE indefinido en cualquier procesador generará una interrupción de este tipo, ya que al intentar decodificarla la CPU no puede generar ninguna acción interna.

¿Qué es una interrupción?

Cuando se genera una interrupción el proceso en ejecución se detiene y «salta» a atender la llamada a interrupción (subrutina).

La ISR se ejecuta y termina (regresa al proceso pausado) con una instrucción de retorno de interrupción IRET, el proceso pausado continua desde donde ocurrió la interrupción.



Interrupciones vs System Calls

- Las **interrupciones** son generadas por eventos externos o internos, como temporizadores, dispositivos periféricos o errores de hardware. Las **llamadas al sistema** son invocadas mediante instrucciones específicas (por ejemplo, syscall en ARM) que **transfieren el control al sistema operativo**.
- Las **interrupciones** pueden ocurrir en cualquier modo de ejecución del procesador y requieren que el sistema operativo responda para manejar el evento. **Las llamadas al sistema** generalmente se realizan desde el modo de usuario y requieren un cambio de contexto al modo kernel, donde el sistema operativo tiene acceso a recursos privilegiados.
- Las **interrupciones** pueden gestionar una variedad de eventos, como solicitudes de E/S, temporizadores, errores de hardware, etc. **Las llamadas al sistema** proporcionan acceso controlado a servicios del sistema operativo, como operaciones de E/S, creación de procesos, gestión de memoria, entre otros.

En conclusión

Mientras que las **interrupciones** son eventos asincrónicos que requieren una respuesta del sistema operativo, las **llamadas al sistema (SysCall)** son solicitudes deliberadas realizadas por programas para acceder a servicios y funciones del sistema operativo.

En un Syscall en Raspbian, un programa llama al núcleo de Linux solicitando un servicio.

Fin, ahora ARM



Registros utilizados

- r0: entrada/salida
- r1: dirección de buffer, cadena
- r2: número de caracteres a leer/mostrar
- r7: syscall

Salida por consola o terminal

mov r7, #4	// Salida por pantalla
mov r0, #1	// salida cadena
mov r2, #30	// Tamaño de la cadena
ldr r1, =mensaje	
swi 0	// SWI, Software interrupt

Leer por teclado

```
mov r7, #3      // Lectura x teclado
mov r0, #0      // Ingreso de cadena
mov r2, #4      // Leer cant caracteres
ldr r1, =cadena // donde se guarda lo ingresado
swi 0           // SWI, Software interrupt
```

Llamada al sistema operativo SysCall

```
mov r7, #1    // Salida al sistema
```

```
swi 0         // svc (supervisor call) es equivalente
```

Introducción al manejo de archivos

Intro Archivos

Los archivos informáticos son estructuras de datos utilizadas para almacenar información de manera persistente en dispositivos de almacenamiento como discos duros, unidades de estado sólido (SSD) o dispositivos de almacenamiento externo.

Los archivos se almacenan en dispositivos de almacenamiento en forma de secuencias de bits organizadas en bloques de datos. Cada archivo está asociado con un nombre único que lo identifica dentro del sistema de archivos. Los sistemas operativos utilizan una tabla de asignación de archivos para realizar un seguimiento de la ubicación física de los archivos en el dispositivo de almacenamiento.

Intro Archivos

Los sistemas operativos proporcionan una variedad de operaciones para manipular archivos, como abrir, leer, escribir y cerrar, que permiten a los programas acceder y gestionar la información almacenada en ellos de manera eficiente

En ensamblador ARM trabajamos con archivos ASCII, para hacerlo bien debemos seguir el siguiente orden:

- **Abrir archivo**
- **leer/modificar texto del archivo**
- **cerrar archivo**

Registros

- r0: Puntero al nombre del archivo
- r1: Lectura o escritura
- r2: para cantidad de caracteres
- r7: syscall para abrir archivo

Lectura: Apertura de archivo

.data

```
/* Definición de datos */
```

```
archi: .asciz "nombrearchivo.txt" // manejamos archivos de texto ".txt"
```

```
buffer: .space 10 // espacio para poner contenido archivo
```

```
buffer2: .ascii "Skynet es ARM" // por si luego queremos escribir nuevo contenido
```

Apertura de archivo para **Lectura**

// Abrir archivo para lectura CUIDADO: ¡El archivo debe existir!

```
mov r7, #5           // Abrir archivo
ldr r0, =archi       // puntero al nombre del archivo
                     // en r0 devuelve el descriptor
mov r1, #0           // #0 para lectura; #2 escritura
mov r2, #0           // cantidad de caracteres, en este caso no lo uso.
swi 0                // SysCall
```

// vemos cómo resultó

```
cmp r0, #0           // ¿podimos abrir el archivo?
blt error            // si es menor, negativo, hubo error
mov r6, r0           // guardo el descriptor en r6
```

Apertura de archivo para **Escritura**

// Abrir archivo para lectura CUIDADO: ¡El archivo debe existir!

```
mov r7, #5           // Abrir archivo
ldr r0, =archi       // puntero al nombre del archivo
                     // en r0 devuelve el descriptor

mov r1, #2           // #2 escritura
mov r2, #438         // permisos para modificar el archivo
swi 0                // Llamada al sistema. SysCall
```

// vemos cómo resultó

```
cmp r0, #0           // ¿podimos abrir el archivo?
blt error            // si es menor, negativo, hubo error
mov r6, r0           // guardo el descriptor en r6
```

Proceso de lectura del archivo

// Leemos archivo

mov r7, #3	// lectura del archivo, porque r0 contiene el descriptor
ldr r1, =buffer	// donde poner el contenido del archivo leído
mov r2, #7	// cantidad de caracteres a leer
swi 0	// Llamada al sistema. SysCall

Escritura de archivos

// modificamos el contenido de archivo. Write archivo

```
mov r0, r6      // Nos aseguramos que r0 contenga descriptor
mov r7, #4      // Escritura, write
ldr r1, =buffer2 // puntero a memoria donde está el nuevo texto
mov r2, #16     // cantidad de caracteres a escribir
swi 0           // Llamada al sistema. SysCall
```

Por último y muy importante o sino...

// Cerramos el archivo abierto

mov r0, r6	// ponemos en r0 descriptor
mov r7, #6	// Cerramos archivo abierto
swi 0	// Llamada al sistema. SysCall

¿Preguntas?